



Fundamentos de entrada/saída e NIO2

- Streams I/O
- NIO2



Entrada/Saída (I/O)

A classe `java.io` trata de leitura e escrita de ficheiros. Permite-nos ir buscar e guardar informações entre diferentes vezes que corremos o nosso programa (como um jogo guardado para mais tarde continuar), bem como facilitar e automatizar o Input de informação.

Temos várias Streams à disposição para diferentes usos. Estas são:

- Byte Streams - dados binários não tratados
- Character Streams - I/O de dados de caracteres, tratando automaticamente tradução de e para o conjunto de caracteres local
- Buffered Streams - otimizam I/O, reduzindo as chamadas à API nativa
- Scanner and Format - permitem ler e escrever informação legível a humanos mais facilmente
- I/O da linha de comandos - descreve as Streams standardizadas e o objeto consola.
- Data Streams - trata do I/O binário de dados primitivos e valores de Strings
- Object Streams - trata I/O binário de Objetos



Byte Streams

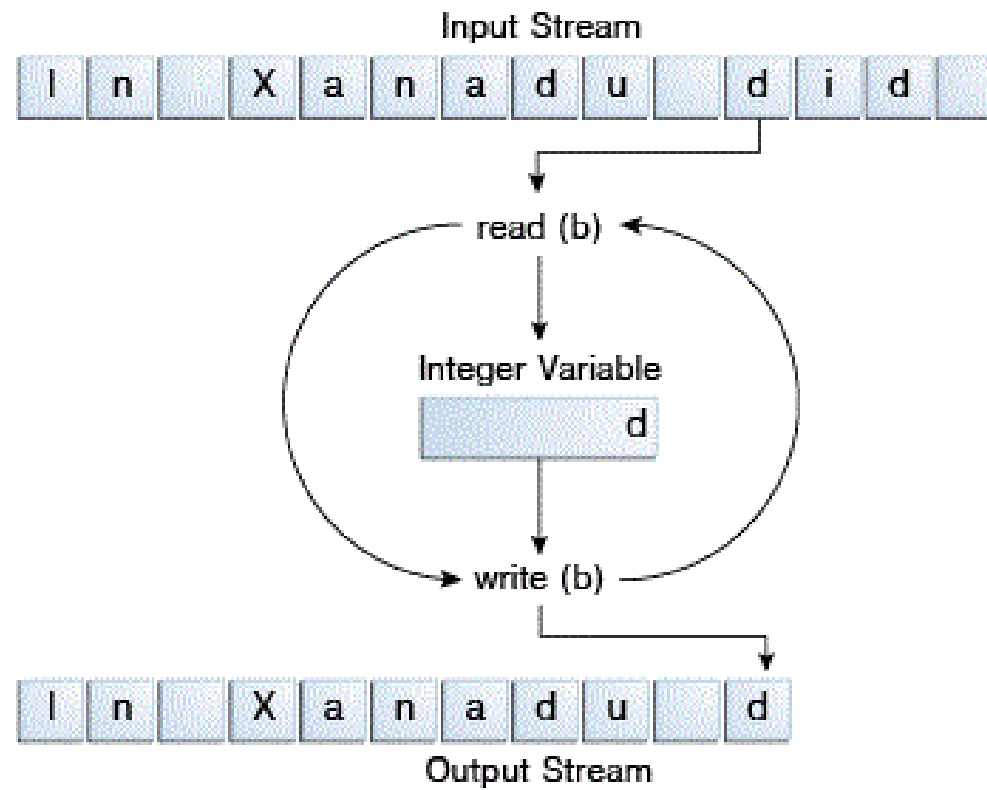
Usadas para tratar informação organizada em Bytes. Todas estas classes herdam de InputStream e OutputStream.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class CopiarBytes{
    public static void main(String[] args) throws IOException {
        FileInputStream ficheiroEntrada= null;
        FileOutputStream ficheiroSaida= null;
        try {
            ficheiroEntrada = new FileInputStream("entrada.txt");
            ficheiroSaida = new FileOutputStream("saida.txt");
            int c;
            while ((c = ficheiroEntrada.read()) != -1) {
                ficheiroSaida.write(c);
            }
        } finally {
            if (ficheiroEntrada != null) {
                ficheiroEntrada.close();
            }
            if (out != null) {
                out.close();
            }
        }
    }
}
```



Byte Streams





Character Streams

Usadas para tratar caracteres. Traduz automaticamente entre os diferentes conjuntos de caracteres, mantendo a legibilidade do texto.

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class CopiarCaracteres{
    public static void main(String[] args) throws IOException {

        FileReader streamEntrada= null;
        FileWriter streamSaida= null;

        try {
            streamEntrada= new FileReader("entrada.txt");
            streamSaida= new FileWriter("characteroutput.txt");

            int c;

            while ((c = streamEntrada.read()) != -1) {
                streamSaida.write(c);
            }
        } finally {
            if (streamEntrada != null) {
                streamEntrada.close();
            }
            if (streamSaida != null) {
                streamSaida.close();
            }
        }
    }
}
```



Character Streams

Também podemos usar com linhas em vez de caracteres

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class CopiarCaracteres{
    public static void main(String[] args) throws IOException {

        FileReader streamEntrada= null;
        FileWriter streamSaida= null;

        try {
            streamEntrada= new FileReader("entrada.txt");
            streamSaida= new FileWriter("characteroutput.txt");

            int l;
```

```
            while ((l = streamEntrada.readLine()) != -1) {
                streamSaida.println(l);
            }
        } finally {
            if (streamEntrada != null) {
                streamEntrada.close();
            }
            if (streamSaida != null) {
                streamSaida.close();
            }
        }
    }
}
```



Buffered Streams

O que vimos até agora são streams unbuffered. Isto significa que cada chamada à API é tratada pelo Sistema Operativo, tornando-se pouco eficiente. As chamadas vão gerar acesso a disco, rede ou uma operação semelhante que vai demorar.

Para reduzir este overhead, a plataforma de Java implementou buffered I/O Streams. Estes lêem (ou escrevem) dados da memória (buffer), sendo o acesso a disco/rede apenas feito quando o buffer estiver vazio (ou cheio no caso da escrita).

```
streamEntrada= new BufferedReader(new FileReader("entrada.txt"));
```

```
streamSaida = new BufferedWriter(new FileWriter("saida.txt"));
```



Buffered Streams

Por vezes podemos querer fazer a escrita de forma imediata, sem esperar que o buffer fique cheio. A isto chama-se flushing. Algumas classes de buffered output suportam autoflush, especificado no construtor através de um argumento opcional.

Para fazermos flush manualmente, basta invocar o método flush em qualquer stream de saída, mas só terá efeito em buffered streams, uma vez que as unbuffered fazem as suas chamadas automaticamente.



Linha de comandos

Muitas vezes, os programas são corridos através da linha de comandos e interagem com o utilizador pela mesma. O Java suporta estas interações de 2 formas: Standard Streams e diretamente pela consola.

As Standard Streams são uma feature de muitos sistemas operativos, lendo o input do teclado e escrevendo no monitor. Também suportam fazê-lo por ficheiros ou mesmo entre diferentes programas, mas isso é controlado pelo interpretador da linha de comandos, não pelos programas.

Sendo disponibilizadas pelo SO, não precisam de ser abertas. Podem ser utilizadas através de `System.in`, `System.out` e `System.err`.

Por motivos históricos, as Standard Streams são Byte Streams.



Linha de comandos

A Consola é uma alternativa mais poderosa que as Standard Streams. A consola é especialmente útil para tratar passwords de forma segura. Também disponibiliza Streams de I/O que são Character Streams. Antes de podermos utilizar a Consola, precisamos de a invocar com `System.console()`. Se estiver disponível, é retornada através deste método. Se retornar NULL, operações de consola não são permitidas ou suportadas pelo SO, ou o programa foi lançado num ambiente não interativo.

A Consola disponibiliza a função `readPassword`. Este método desliga o eco do Input e retorna um array de caracteres em vez de uma String, fazendo com que seja apagada da memória assim que não for mais necessária.



Linha de comandos

```
import java.io.Console;

import java.util.Arrays;

import java.io.IOException;

public class Password {

    public static void main (String args[]) throws IOException {

        Console c = System.console();

        if (c == null) {

            System.err.println("Sem consola.");

            System.exit(1);

        }

        String login = c.readLine("Enter your login: ");

        char [] passwordAntiga = c.readPassword("Introduz a password antiga: ");
```

```
        if (verify(login, passwordAntiga)) {

            boolean naoCoincide;

            do {

                char [] novaPassword1 = c.readPassword("Introduz a password nova: ");

                char [] novaPassword2 = c.readPassword("Confirma a password nova: ");

                noMatch = ! Arrays.equals(novaPassword1, novaPassword2);

                if (naoCoincide) {

                    c.format("As passwords não coincidem, tenta novamente.%n");

                } else {

                    change(login, novaPassword1);

                    c.format("A password para o utilizador %s foi alterada.%n", login);

                }

                Arrays.fill(novaPassword1, ' ');

                Arrays.fill(novaPassword2, ' ');

            } while (naoCoincide);

        }

        Arrays.fill(passwordAntiga, ' ');

    }

}
```



Data Streams

Permitem ler e escrever variáveis dos tipos primitivos (boolean, char, byte, short, int, long, float e double), bem como Strings.

```
static final String dataFile = "dadosFaturacao";
```

```
static final double[] precos= { 19.99, 9.99, 15.99, 3.99, 4.99 };
```

```
static final int[] unidades = { 12, 8, 13, 29, 50 };
```

```
static final String[] descricoes = {
```

```
    "T-shirt Java",
```

```
    "Caneca Unity",
```

```
    "Autocolantes Python",
```

```
    "Porta-chaves Unity",
```

```
    "Iman Python"
```

```
};
```

```
saida = new DataOutputStream(new BufferedOutputStream(  
    new FileOutputStream(ficheiroDados)));
```

```
for (int i = 0; i < precos.length; i++) {  
    out.writeDouble(precos[i]);  
    out.writeInt(unidades[i]);  
    out.writeUTF(descricoes[i]);  
}
```

```
entrada = new DataInputStream(new  
    BufferedInputStream(new  
    FileInputStream(ficheiroDados)));
```

```
double preco;
```

```
int quantidade;
```

```
String desc;
```

```
double total = 0.0;
```

```
try {
```

```
    while (true) {
```

```
        precos = in.readDouble();
```

```
        quantidade = in.readInt();
```

```
        desc = in.readUTF();
```

```
        System.out.format("Compraste %d" + "  
                            unidades de %s a $%.2f%n",
```

```
                            quantidade, descricao, preco);
```

```
        total += quantidade * preco;
```

```
    }
```

```
} catch (EOFException e) {
```

```
}
```

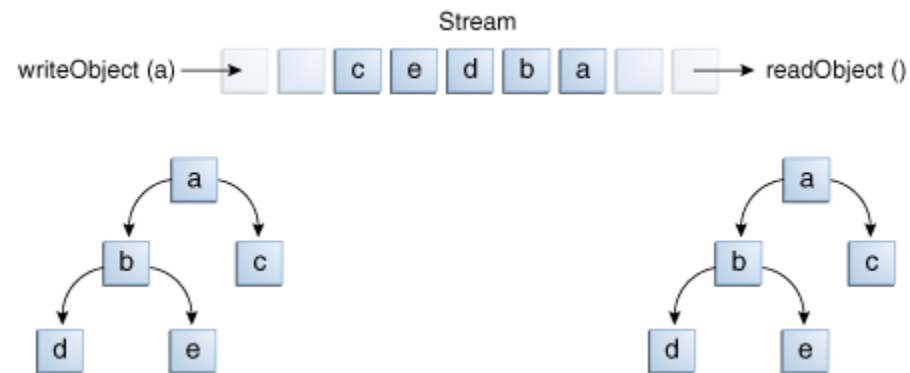


Object Streams

Estas streams permitem serializar objetos. A maioria das classes standard suporta serialização. As que suportam, implementam a interface `Serializable`.

Estas streams são `ObjectInputStream` e `ObjectOutputStream`. Estas classes implementam `ObjectInput` e `ObjectOutput`, subinterfaces com `DataInput`. Isto significa que os dados primitivos que suportam Data Streams também irão suportar Object Streams. Uma Object Stream pode conter uma mistura de valores primitivos e objetos.

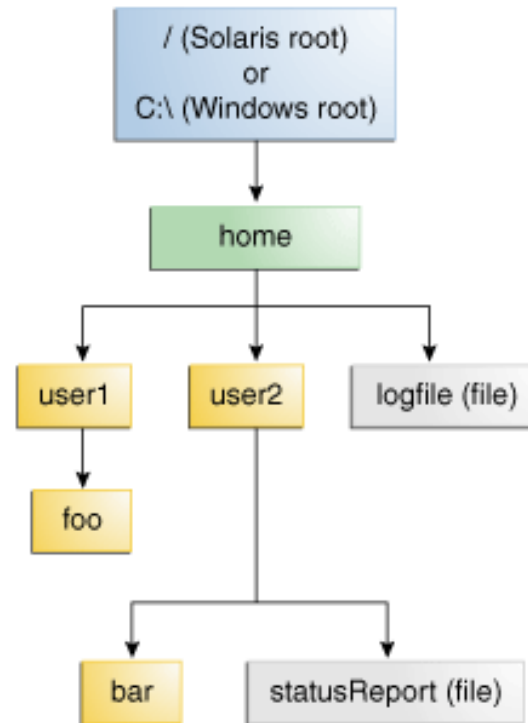
```
Object ob = new Object();  
out.writeObject(ob);  
out.writeObject(ob);
```



```
Object ob1 = in.readObject();  
Object ob2 = in.readObject();
```



O que é um Path?





O que podemos fazer com um path?

```
System.out.format("toString: %s\n", path.toString());
System.out.format("getFileName: %s\n", path.getFileName());
System.out.format("getName(0): %s\n", path.getName(0));
System.out.format("getNameCount: %d\n", path.getNameCount());
System.out.format("subpath(0,2): %s\n", path.subpath(0,2));
System.out.format("getParent: %s\n", path.getParent());
System.out.format("getRoot: %s\n", path.getRoot());
Path p1 = Paths.get("C:\\home\\joe\\foo");
// Result is C:\home\joe\foo\bar
System.out.format("%s\n", p1.resolve("bar"));
```

```
Path p1 = Paths.get("joe");
Path p2 = Paths.get("sally");
// Result is ../sally
Path p1_para_p2 = p1.relativize(p2);
// Result is ../joe
Path p2_para_p1 = p2.relativize(p1);
```

```
Path p1 = Paths.get("home");
Path p3 = Paths.get("home/sally/bar");
// Result is sally/bar
Path p1_para_p3 = p1.relativize(p3);
// Result is ../..
Path p3_para_p1 = p3.relativize(p1);
```

```
Path path = ...;
Path outroPath = ...;
Path inicio = Paths.get("/home");
Path fim = Paths.get("foo");

if (path.equals(outroPath)) {
    // logica igualdade
} else if (path.startsWith(inicio)) {
    // path começa com "/home"
} else if (path.endsWith(fim)) {
    // path acaba com "foo"
}
```



Como verificar ficheiros

Ao verificarmos a existência (ou não existência de um ficheiro, temos 3 resultados possíveis:

- Existe
- Não existe
- Desconhecido

Podemos utilizar as mesmas funções que para um Path: `Files.exists(path)`, `Files.notExists(path)`.

Podemos verificar se os conseguimos aceder com `isReadable(Path)`, `isWritable(Path)`, e `isExecutable(Path)`

Podemos verificar se dois Paths têm o mesmo ficheiro com o seguinte código:

```
Path p1 = ...;
```

```
Path p2 = ...;
```

```
if (Files.isSameFile(p1, p2)) {
```

```
    //Logica para quando os ficheiros são o mesmo
```

```
}
```




Como apagar ficheiros

```
try {  
    Files.delete(path);  
} catch (NoSuchFileException x) {  
    System.err.format("%s: O ficheiro ou diretório não existe", path);  
} catch (DirectoryNotEmptyException x) {  
    System.err.format("%s não está vazio%n", path);  
} catch (IOException x) {  
    // Problemas de permissões são apanhados aqui  
    System.err.println(x);  
}
```



Como copiar ficheiros

Podemos copiar um ficheiro da seguinte forma:

```
import static java.nio.file.StandardCopyOption.*;  
  
...  
  
Files.copy(origem, destino, REPLACE_EXISTING);
```

Existem 3 opções que podemos fornecer na cópia:

- `REPLACE_EXISTING` - substitui se no destino já existir ficheiro com o nome
- `COPY_ATTRIBUTES` - copia os atributos associados ao ficheiro origem para o ficheiro destino.
- `NOFOLLOW_LINKS` - indica que os links simbólicos não devem ser copiados (copia o ficheiro em vez do link)



Como mover ficheiros

Podemos mover um ficheiro da seguinte forma:

```
import static java.nio.file.StandardCopyOption.*;  
  
...  
  
Files.move(origem, destino, REPLACE_EXISTING);
```

Existem 2 opções que podemos fornecer ao mover:

- REPLACE_EXISTING - substitui se no destino já existir ficheiro com o nome
- ATOMIC_MOVE - move o ficheiro de uma vez, garantindo que qualquer programa só verá o ficheiro quando estiver pronto



Ficheiros de Acesso Aleatório

```
String s = "I was here!\n";
```

```
byte data[] = s.getBytes();
```

```
ByteBuffer out = ByteBuffer.wrap(data);
```

```
ByteBuffer copy = ByteBuffer.allocate(12);
```

```
try (FileChannel fc = (FileChannel.open(file, READ, WRITE))) {
```

```
    // Read the first 12
```

```
    // bytes of the file.
```

```
    int nread;
```

```
    do {
```

```
        nread = fc.read(copy);
```

```
    } while (nread != -1 && copy.hasRemaining());
```

```
    // Write "I was here!" at the beginning of the file.
```

```
    fc.position(0);
```

```
    while (out.hasRemaining())
```

```
        fc.write(out);
```

```
    out.rewind();
```

```
    // Move to the end of the file. Copy the first 12 bytes to
```

```
    // the end of the file. Then write "I was here!" again.
```

```
    long length = fc.size();
```

```
    fc.position(length-1);
```

```
    copy.flip();
```

```
    while (copy.hasRemaining())
```

```
        fc.write(copy);
```

```
    while (out.hasRemaining())
```

```
        fc.write(out);
```

```
    } catch (IOException x) {
```

```
        System.out.println("I/O Exception: " + x);
```

```
    }
```



Como criar e ler diretórios

```
Iterable<Path> dirs = FileSystems.getDefault().getRootDirectories();

for (Path name: dirs) {
    System.err.println(name);
}

Set<PosixFilePermission> perms = PosixFilePermissions.fromString("rwxr-x---");
FileAttribute<Set<PosixFilePermission>> attr = PosixFilePermissions.asFileAttribute(perms);
Files.createDirectory(file, attr);

Files.createDirectories(Paths.get("foo/bar/test"));
```

```
Path dir = ...;

try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir)) {
    for (Path file: stream) {
        System.out.println(file.getFileName());
    }
} catch (IOException | DirectoryIteratorException x) {
    // IOException can never be thrown by the iteration.
    // In this snippet, it can only be thrown by newDirectoryStream.

    System.err.println(x);
}
```



Hard Links e Symbolic Links

```
Path newLink = ...;

Path existingFile = ...;

try {
    Files.createLink(newLink, existingFile);
} catch (IOException x) {
    System.err.println(x);
} catch (UnsupportedOperationException x) {
    // Some file systems do not
    // support adding an existing
    // file to a directory.
    System.err.println(x);
}
```

```
Path newLink = ...;

Path target = ...;

try {
    Files.createSymbolicLink(newLink, target);
} catch (IOException x) {
    System.err.println(x);
} catch (UnsupportedOperationException x) {
    // Some file systems do not support symbolic links.
    System.err.println(x);
}
```



Symbolic Links

```
Path file = ...;  
  
boolean isSymbolicLink =  
    Files.isSymbolicLink(file);
```

```
Path link = ...;  
  
try {  
    System.out.format("Target of link" +  
        " '%s' is '%s'%n", link,  
        Files.readSymbolicLink(link));  
} catch (IOException x) {  
    System.err.println(x);  
}
```



Navegar diretórios

```
import static java.nio.file.FileVisitResult.*;

public static class PrintFiles

    extends SimpleFileVisitor<Path> {

    // Print information about each type of file.

    @Override

    public FileVisitResult visitFile(Path file,

                                    BasicFileAttributes attr) {

        if (attr.isSymbolicLink()) {

            System.out.format("Symbolic link: %s ", file);

        } else if (attr.isRegularFile()) {

            System.out.format("Regular file: %s ", file);

        } else {

            System.out.format("Other: %s ", file);

        }

        System.out.println("(" + attr.size() + "bytes)");

        return CONTINUE;

    }

}
```

```
// Print each directory visited.
```

```
@Override
```

```
public FileVisitResult postVisitDirectory(Path dir,
```

```
                                           IOException exc) {
```

```
    System.out.format("Directory: %s%n", dir);
```

```
    return CONTINUE;
```

```
}
```

```
// If there is some error accessing
```

```
// the file, let the user know.
```

```
// If you don't override this method
```

```
// and an error occurs, an IOException
```

```
// is thrown.
```

```
@Override
```

```
public FileVisitResult visitFileFailed(Path file, IOException exc) {
```

```
    System.err.println(exc);
```

```
    return CONTINUE;
```

```
}
```

```
}
```




Vigiar diretórios

```
import static java.nio.file.StandardWatchEventKinds.*;
```

```
Path dir = ...;
```

```
try {
```

```
    WatchKey key = dir.register(watcher,
```

```
        ENTRY_CREATE,
```

```
        ENTRY_DELETE,
```

```
        ENTRY_MODIFY);
```

```
    } catch (IOException x) {
```

```
        System.err.println(x);
```

```
    }
```



Dúvidas?

Tokio.